

🔗 本章目标

把模板理解成代码生成器:函数模板与类模板如何实例化,特化与偏特化如何定制行为,可变参数与折叠表达式如何展开;掌握 SFINAE、enable_if 与 C++20 概念(concepts);用 constexpr 在编译期完成计算。理解模板与 C 宏、Java 泛型的本质区别。

📦 前置知识

熟练使用 std::vector、std::string 等模板容器(第1章起一直在用);了解函数重载(第4章)。本章代码基于 C++17,concepts 部分按 C++20 语法展示。模板错误信息长是常态,先看第一个错误行。

\$10.1 模板是代码生成器	\$10.2 函数模板与推导	\$10.3 类模板与特化
\$10.4 可变参数模板	\$10.5 SFINAE 与 concepts	\$10.6 constexpr 元编程

\$10.1 模板是代码生成器

C 的泛型工具只有宏与 void*:宏是纯文本替换,没有类型检查,还有双重求值这类暗坑,void* 丢掉类型信息,一切类型错误推迟到运行时。C++ 的模板走完全不同的路线:模板是代码生成器——写一份 template,编译器为每个用到的类型生成一份独立代码。std::vector<int> 和 std::vector<double> 是两份不同的类,各自的成员函数都是独立的机器码,类型检查在生成时完成,运行期零抽象开销。

理解"生成"二字,模板的许多现象就都说得通了:为什么模板通常要整体放在头文件里——因为每个编译单元都需要看到模板定义才能生成代码,这与普通函数"声明放头文件、定义放源文件"的纪律不同;为什么模板实例化越多编译越慢——生成代码也是要花时间的,工程上要节制模板的使用面。与之对照,Java 的泛型是擦除的:编译后只有一份代码,类型参数被替换成 Object,运行期靠强制转换与检查,基本类型还得装箱;C++ 是"每个类型生成一份代码,类型信息保留到运行期之前",因此 std::vector<int> 与原生数组一样快,这是模板在 C++ 里全面取代宏与 void* 的根本原因。

C 的写法	C++ 的写法	说明
#define MAX(a,b) ((a)>(b)?(a):(b))	template<typename T> T maxOf(T a, T b)	宏是文本替换,C++ 类型安全
宏双重求值:MAX(i++, j++)	模板参数求值一次	宏的经典暗坑,模板没有
void* 传数据,丢失类型	每种类型生成独立代码	错误留在编译期,运行期零开销

清单 10-1 函数模板:第一个模板

```

#include <iostream>
#include <string>

// 模板:一份定义,为每个 T 生成一份代码
template <typename T>
T maxOf(T a, T b) {
    return a > b ? a : b;
}

int main() {
    int m1 = maxOf(3, 7);           // 推导 T = int
    double m2 = maxOf(2.5, 1.5);    // 推导 T = double
    std::string m3 = maxOf(std::string("甲"), std::string("乙"));
    std::cout << m1 << " " << m2 << " " << m3 << "\n";
    // 上面三行,编译器实际生成了三个不同版本的 maxOf
    return 0;
}

```

§10.2 模板参数与推导

模板参数不只是类型:它可以是类型(`typename T`)、非类型值(`int N`)与模板模板参数。非类型参数让模板拥有“编译期常量”能力:`std::array<int, 5>` 的 5 就是非类型参数,数组长度在编译期就固定并写进类型,越界检查不再需要运行时开销。调用时,函数模板从实参自动推导类型参数;推导不出(比如两个参数类型不同)就显式指定:`maxOf<double>(3, 2)`。推导遵循一套精确规则,最常见的原则是:按值传参时数组与函数会退化成指针,引用传参时保留完整类型——`std::forward` 的完美转发正是靠这条规则实现的。

推导失败或显式指定错误都会在实例化时报错,错误信息长而费解,这是模板的学费。工程应对:先从第一个错误行开始读(往往是“找不到匹配的调用”或“模板实参不匹配”),再看它引用的模板定义;复杂模板库(如标准库算法)的错误,先怀疑自己的类型不满足该算法要求的操作——`vector` 传给了需要排序的接口,通常就是缺少某个运算符。C++20 的 `concepts`(§10.5)用约束把这类错误提前到函数签名处,信息质量大幅改善。

清单 10-2 非类型参数与显式指定

```

#include <iostream>
#include <array>

// 非类型参数:N 是编译期常量
template <typename T, int N>
T sumOf(const std::array<T, N>& a) {
    T s{};
    for (int i = 0; i < N; ++i) s += a[i];
    return s;
}

int main() {
    std::array<int, 3> x{1, 2, 3};      // N = 3 写进类型
    std::array<double, 2> y{1.5, 2.5};
    std::cout << sumOf(x) << " " << sumOf(y) << "\n";    // 6 4
    std::cout << sumOf<int, 3>(x) << "\n";    // 显式指定参数
    return 0;
}

```

§10.3 类模板、特化与偏特化

类模板与函数模板同理:template<typename T> class Box { ... },成员函数在使用到的方法内按需实例化。写一个最简单的 Box 立刻就能体会:数据成员 T value,get 与 set 的类型随 T 变化,而成员函数只有被调用时才生成代码——没调用 set 的 Box 不会生成 set 的机器码。这让模板成为"按需定制"的机制,而不是一揽子复制。

特化是模板家族最强大的定制工具,也是 Java 泛型完全没有的能力:全特化 template<> class Box<bool> 为特定类型提供专属实现;偏特化 template<typename T> class Box<T*> 为一族类型(所有指针)提供实现。标准库大量使用特化:std::vector<bool> 用位压缩存储、std::hash 为每个类型定制哈希——编译器选择"最匹配"的特化版本,这和重载决议是同一思想,只是发生在类型层面。写特化时注意:特化不能出现在首次使用之后(一般紧跟主模板声明),这是 C++ 对"代码生成顺序"的硬性要求。

C 的写法	C++ 的写法	说明
为每种类型复制粘贴一份结构体	一份类模板 + 特化	特化按需定制,不复制整体
无对应能力	template<> class Box<bool>	全特化:单一类型专属实现
无对应能力	template<typename T> class Box<T*>	偏特化:一族类型的专属实现

清单 10-3 类模板与特化

```

#include <iostream>
#include <string>

template <typename T>
class Box {
public:
    explicit Box(T v) : value(v) {}
    T get() const { return value; }
    void set(T v) { value = v; }
    std::string describe() const { return "普通盒"; }
private:
    T value;
};

// 全特化:bool 版用 1 字节,输出专属描述
template <>
class Box<bool> {
public:
    explicit Box(bool v) : value(v) {}
    bool get() const { return value; }
    void set(bool v) { value = v; }
    std::string describe() const { return value ? "真" : "假"; }
private:
    bool value;
};

// 偏特化:指针版,get 返回解引用后的值
template <typename T>
class Box<T*> {
public:
    explicit Box(T* p) : ptr(p) {}
    T get() const { return *ptr; }
    void set(T v) { *ptr = v; }
    std::string describe() const { return "指针盒"; }
private:
    T* ptr;
};

int main() {
    Box<int> a(42);
    Box<bool> b(true);           // 命中全特化
    int raw = 7;
    Box<int*> c(&raw);           // 命中偏特化
    std::cout << a.describe() << " " << b.describe() << " "
              << c.describe() << "\n";
    return 0;
}

```

§10.4 可变参数模板与折叠表达式

可变参数模板让函数接受任意数量的任意类型参数:template<typename... Args>,参数包 Args... 在展开点逐个展开。打印任意多个值是经典入门:递归展开(取第一个参数,剩余继续递归)是 C++11 的写法;C++17 的折叠表达式把常见模

式压缩成一行:(args + ...) 求全部参数的和,折叠运算符可以是任何二元运算符。printf 式的格式化与 std::tuple、std::variant 的底层都是参数包。

折叠表达式有三种形态需要分清:一元右折叠 (args + ...) 等价 a + (b + c);一元左折叠 (... + args) 等价 (a + b) + c ——加法无所谓,减法等非结合运算结果不同;二元折叠 (0 + ... + args) 提供初始值。参数包还可以与 sizeof... 配合查询个数,与转发引用配合实现完美转发。学会把"任意多参数"的接口交给参数包,你的函数签名就从固定参数表解放出来。

清单 10-4 参数包与折叠表达式

```
#include <iostream>
#include <string>

// 递归展开:每次取第一个参数,剩余继续
template <typename T>
void printAll(const T& v) {                // 递归终点:只剩一个
    std::cout << v << "\n";
}

template <typename T, typename... Rest>
void printAll(const T& v, const Rest&... rest) {
    std::cout << v << " ";
    printAll(rest...);                    // 展开剩余参数
}

// C++17 折叠表达式:一行求所有参数之和
template <typename... Args>
auto sumAll(const Args&... args) {
    return (args + ...);                 // 一元右折叠
}

int main() {
    printAll(1, 2.5, "三", std::string("四"));
    std::cout << "个数:" << sizeof...(int) << "\n";    // 语法演示
    std::cout << sumAll(1, 2, 3, 4) << " " << sumAll(1.5, 2.5) << "\n";
    return 0;
}
```

§10.5 SFINAE、enable_if 与 concepts

SFINAE(Substitution Failure Is Not An Error,替换失败不是错误)是模板世界的"条件编译":实例化时,某个模板替换出的声明不合法(如类型没有某种运算符),编译器不把它当错误,而是把这个候选从重载集合中移除,继续找别的匹配。基于它,C++11 的 std::enable_if 实现"条件启用":模板参数满足条件才存在,不满足就像没声明过一样。经典用法是给"只接受数值类型"的函数加约束:std::enable_if<std::is_arithmetic_v<T>>::type 作为返回类型或默认模板参数。

C++20 的 concepts 把 SFINAE 的繁琐压缩成自然语言:概念(concept)是一组约束的命名,如 std::integral、std::floating_point、std::ranges::range;requires 子句或 requires 表达式声明约束:template<typename T> requires std::integral<T> void f(T);。概念约束错误发生在函数签名处,报错信息直接说"不满足 integral",而不是一屏模板展开。工程建议:新代码优先 concepts,老代码维持 enable_if;两者背后是同一套替换机制。

C 的写法	C++ 的写法	说明
宏 #ifdef 条件编译	enable_if / concepts 约束	宏是文本级条件,C++ 是类型级约束
传错类型编译不过但报错玄妙	requires std::integral<T> 直说	concepts 让错误信息可读
无对应能力	if constexpr 编译期分支	按类型特性裁剪代码路径

清单 10-5 enable_if 与 concepts

```
#include <iostream>
#include <type_traits>
#include <string>
#include <concepts>      // C++20

// enable_if 写法(C++11):只接受算术类型
template <typename T>
typename std::enable_if<std::is_arithmetic_v<T>, T>::type
twice(T v) {
    return v * 2;
}

// concepts 写法(C++20):约束写进签名
template <std::integral T>      // 只接受整型
T triple(T v) {
    return v * 3;
}

template <typename T>
requires std::is_arithmetic_v<T>    // requires 子句
T plusOne(T v) {
    return v + 1;
}

int main() {
    std::cout << twice(21) << " " << twice(2.5) << "\n";    // 42 5
    std::cout << triple(14) << "\n";                        // 42
    std::cout << plusOne(41) << "\n";                        // 42
    // std::cout << twice(std::string("x"));    // 错误:不满足约束
    return 0;
}
```

⚠ 易错点 1:模板定义必须在使用前可见

模板的“代码生成”性质决定了它不能像普通函数那样声明与定义分离:定义在 .cpp 里的函数模板,另一个 .cpp 里调用会直接链接失败(找不到实例)。规则:函数模板与类模板的完整定义放头文件;确要隐藏实现,用显式实例化声明,但只对预定的类型生效。

§10.6 constexpr 与编译期计算

constexpr 函数承诺“能编译期求值就在编译期求值”:只要参数是编译期常量,调用结果就成为常量;参数不是,它就退化为普通函数。constexpr 变量的值在编译期就确定,可以用于模板非类型参数、数组大小等场景。C++14 放宽了

constexpr 函数的限制(允许循环与局部变量),C++17 的 if constexpr 更进一步:按编译期条件裁剪代码分支,不满足的分支根本不会被实例化——这是写“同构算法适配多种类型”的主力工具。

与 C 的宏编程对比,constexpr 的优势是显而易见的:C 用宏做编译期计算,先有 #if 预处理,后有 sizeof 技巧,但宏没有类型、没有作用域、不参与重载决议;constexpr 函数是真正的函数,有类型检查、有调试信息、能被测试。现代 C++ 的编译期计算栈是 constexpr 函数 + if constexpr + 模板,替代了 C 时代的大部分宏魔术,可读性与可维护性都不可同日而语。

清单 10-6 constexpr 函数与 if constexpr

```
#include <iostream>
#include <type_traits>

// constexpr:参数是编译期常量时,结果也是常量
constexpr long long factorial(int n) {
    long long r = 1;
    for (int i = 2; i <= n; ++i) r *= i;    // C++14 起允许循环
    return r;
}

// if constexpr:按类型裁剪分支,不满足的分支不实例化
template <typename T>
auto describe(T v) {
    if constexpr (std::is_integral_v<T>) {
        return std::string("整数 ") + std::to_string(v);
    } else if constexpr (std::is_floating_point_v<T>) {
        return std::string("浮点 ") + std::to_string(v);
    } else {
        return std::string("其他 ") + std::string(v);
    }
}

int main() {
    constexpr long long f = factorial(10);    // 编译期算好
    std::cout << f << "\n";                  // 3628800
    static_assert(f == 3628800, "编译期断言");    // 编译期验证
    std::cout << describe(42) << "\n";
    std::cout << describe(3.14) << "\n";
    std::cout << describe(std::string("文本")) << "\n";
    return 0;
}
```

⚠ 易错点 2:constexpr 不是免费的

constexpr 在编译期递归或循环很深的计算(如 factorial(100000))会让编译器卡死;constexpr 函数体里不能做 I/O、动态内存分配等运行期操作。原则:编译期计算用于小规模常量推导(表、偏移、尺寸),重计算留在运行期。

⚠ 易错点 3:vector<bool> 是特化的陷阱

std::vector<bool> 为节省内存按位存储,元素是代理对象而非真正的 bool 引用:auto b = v[0] 得到代理,改 b 改不到原数组,&v[0] 也取不到 bool 地址。需要真正的布尔数组用 std::vector<char> 或 std::bitset;这个例子提醒我们:特化可以改变语义,读标准库文档比猜行为可靠。

泛型与元编程速查表

概念	语法示例	要点
函数模板	<code>template<typename T> T f(T v)</code>	按实参推导,一份定义多份代码
类模板	<code>template<typename T> class Box</code>	成员函数按需实例化
非类型参数	<code>template<typename T, int N></code>	编译期常量,如 array 长度
全特化	<code>template<> class Box<bool></code>	单一类型专属实现
偏特化	<code>template<typename T> class Box<T*></code>	一族类型的专属实现
参数包	<code>template<typename... Args></code>	任意数量任意类型参数
折叠表达式	<code>(args + ...)</code>	参数包整体运算,注意结合方向
SFINAE	替换失败不是错误	约束候选集,启用条件模板
<code>enable_if</code>	<code>std::enable_if<cond, T>::type</code>	条件启用,老式约束
concepts	<code>template<std::integral T></code>	C++20,约束进签名,报错清晰
constexpr 函数	<code>constexpr int f(int)</code>	编译期可求值,运行期也可用
if constexpr	<code>if constexpr (cond)</code>	编译期分支,不满足的不实例化
<code>static_assert</code>	<code>static_assert(expr, "msg")</code>	编译期断言,失败即编译错误
<code>decltype</code>	<code>decltype(expr)</code>	取表达式类型,配合 auto 使用

自测题

- Q** 1. 模板与 C 宏、Java 泛型在实现上的本质区别是什么?
- Q** 2. 为什么函数模板的定义通常要放在头文件里?
- Q** 3. 全特化与偏特化有何区别?各自举一个标准库中的例子。
- Q** 4. 折叠表达式 `(args - ...)` 与 `(... - args)` 的结果一样吗?为什么?
- Q** 5. SFINAE 与 concepts 解决什么问题?concepts 的优势是什么?

✅ 参考答案

1. C 宏是纯文本替换,无类型检查,有双重求值问题;Java 泛型编译后擦除为一份代码,类型信息丢失,基本类型要装箱;C++ 模板为每个类型实例化生成独立代码,类型检查在编译期,运行期零抽象开销。
2. 模板实例化发生在编译期,每个编译单元都需要看到完整定义才能生成代码;声明与定义分离后,其他编译单元看不到定义,链接时找不到实例。要隐藏实现只能用显式实例化,但只对预定类型生效。
3. 全特化针对单一类型(`template<> class Box<bool>`),如 `std::vector<bool>` 的位压缩;偏特化针对一族类型(`template<typename T> class Box<T*>`),如 `std::hash<T*>` 指针哈希。编译器选择最匹配的特化。
4. 不一样。 $(args - \dots)$ 是右折叠,等价 $a - (b - c)$; $(\dots - args)$ 是左折叠,等价 $(a - b) - c$ 。减法不满足结合律,结果不同;加法、乘法等结合运算则相同。
5. 它们解决"约束模板能接受什么类型":不满足条件的候选从重载集合中移除或直接报错。concepts 把约束写进函数签名(`requires` 子句),报错信息直接指出违反的概念,而 `enable_if` 的报错是长串模板展开,concepts 可读性完胜。

实验:通用统计工具集

本实验用一个小型统计库串联本章全部知识点:函数模板做均值与极值,类模板做固定容量缓冲区,偏特化给指针版缓冲区,`enable_if` 与 `concepts` 约束数值类型,参数包与折叠表达式求总和,`constexpr` 提供编译期常量。代码分三部分,覆盖点已在注释中标明。

实验 10•第一部分:函数模板与数值约束(覆盖:函数模板、`enable_if`、`concepts`、折叠表达式)

```

#include <iostream>
#include <string>
#include <type_traits>
#include <concepts>      // C++20 概念
#include <vector>

// 覆盖:函数模板 + enable_if—只接受算术类型
template <typename T>
typename std::enable_if<std::is_arithmetic_v<T>, T>::type
mean(const std::vector<T>& v) {
    T s{};
    for (const T& x : v) s += x;
    return v.empty() ? T{} : s / (T)v.size();
}

// 覆盖:concepts 约束 + 参数包 + 折叠表达式
template <std::integral... Ts>          // 全部参数都必须是整型
auto sumAll(const Ts&... args) {
    return (args + ...);               // 覆盖:一元右折叠
}

// 覆盖:约束的模板—泛型极值,要求 T 可比较
template <typename T>
requires std::is_arithmetic_v<T>
T maxOf(const std::vector<T>& v) {
    T m = v.front();
    for (const T& x : v) if (x > m) m = x;
    return m;
}

```

实验 10•第二部分:类模板与特化(覆盖:类模板、全特化、偏特化)

```
// 覆盖:类模板—固定容量缓冲区
template <typename T, int N>
class Buffer {
public:
    void push(T v) {
        if (count < N) data[count++] = v;    // 满则丢弃
    }
    T at(int i) const { return data[i]; }
    int size() const { return count; }
    std::string describe() const { return "普通缓冲"; }
private:
    T data[N] = {};
    int count = 0;
};
```

```
// 覆盖:全特化—bool 缓冲,用字节记录
template <int N>
class Buffer<bool, N> {
public:
    void push(bool v) {
        if (count < N) bits[count++] = v;
    }
    bool at(int i) const { return bits[i]; }
    int size() const { return count; }
    std::string describe() const { return "位缓冲"; }
private:
    bool bits[N] = {};
    int count = 0;
};
```

```
// 覆盖:偏特化—指针缓冲,打印地址信息
template <typename T, int N>
class Buffer<T*, N> {
public:
    void push(T* v) {
        if (count < N) data[count++] = v;
    }
    T at(int i) const { return *data[i]; }
    int size() const { return count; }
    std::string describe() const { return "指针缓冲"; }
private:
    T* data[N] = {};
    int count = 0;
};
```

实验 10•第三部分:constexpr 与 main(覆盖:constexpr、if constexpr、static_assert)

```

// 覆盖:constexpr 编译期函数
constexpr int fib(int n) {
    return n <= 1 ? n : fib(n - 1) + fib(n - 2);
}

template <typename T>
std::string kind(const T& v) {
    // 覆盖:if constexpr 编译期分支
    if constexpr (std::is_integral_v<T>) {
        return "整型 " + std::to_string(v);
    } else {
        return "其他 " + std::to_string((double)v);
    }
}

int main() {
    // 覆盖:编译期常量与编译期断言
    constexpr int F10 = fib(10);
    static_assert(F10 == 55, "fib(10) 必须是 55");

    Buffer<int, 4> ib; // 类模板
    ib.push(1); ib.push(2); ib.push(3); ib.push(4); ib.push(5);
    std::cout << ib.describe() << ",大小 " << ib.size() << "\n";

    Buffer<bool, 8> bb; // 全特化
    bb.push(true); bb.push(false);
    std::cout << bb.describe() << " " << bb.at(1) << "\n";

    int raw = 9;
    Buffer<int*, 2> pb; // 偏特化
    pb.push(&raw);
    std::cout << pb.describe() << " " << pb.at(0) << "\n";

    std::vector<double> v{1.5, 2.5, 3.5};
    std::cout << "均值 " << mean(v) << ",最大 " << maxOf(v) << "\n";
    std::cout << "总和 " << sumAll(1, 2, 3, 4) << "\n";
    std::cout << kind(42) << ", " << kind(3.14) << "\n";
    return 0;
}

```

实验覆盖点核对:函数模板(mean/maxOf)、enable_if(mean 的算术约束)、concepts(sumAll 的 integral 约束)、参数包与折叠表达式(sumAll)、类模板(Buffer)、非类型参数(N)、全特化(Buffer<bool>)、偏特化(Buffer<T*>)、constexpr(fib)与 static_assert、if constexpr(kind)。扩展练习:给 Buffer 增加全特化 Buffer<char> 记录字符串;或给 mean 增加 concept 版本,对比两种约束写法的报错信息。



工程建议 1:接口用约束,实现用 if constexpr

对外模板接口用 concepts 或 enable_if 声明"接受什么",内部用 if constexpr 按类型裁剪行为。约束在外,分支在内,调用者看到的签名可读,维护者看到的分支清晰,这是模板工程的黄金分工。

工程建议 2:控制模板规模,警惕编译膨胀

模板是代码生成器,实例化一多编译时间与二进制体积都会膨胀。几十个实例是常态,成千上万个实例就要反思:是否该用虚函数(第6章)把运行时变化隔离,或用类型擦除(如 `std::function`)收敛模板面。能用值语义与虚函数解决的问题,不必全部模板化。

本章小结

模板是类型安全的代码生成器,与 C 宏和 Java 泛型有本质区别;特化与偏特化在类型层面定制行为,参数包与折叠表达式处理任意参数;SFINAE、`enable_if` 与 `concepts` 让约束成为签名的一部分;`constexpr` 把计算搬到编译期。下一章讲"没有反射的 C++":RTTI 与动态特性。